

Network Protocol Stack Simulator

Specification & Submission Report

Name	Enroll No.
Priyanshu Dayal	2023BITE056
Gulaba Ram	2023BITE084
Pranav Abani	2023BITE078

GitHub Repository: [network_protocol_simulator](#)

Language & Libraries Used

Language:	Python 3 — Object-oriented and procedural paradigms.
Standard Library:	random, collections, enum — Used for MAC address generation, queue management, and s
Data Structures:	Python built-in list and dictionary — For connection tables, MAC tables, and packet queues.
I/O Representation:	Console-based text output with structured print statements for clear protocol-level tracing.

Project Overview

This project implements a fully object-oriented **Network Protocol Stack Simulator** in Python. The simulator models networking concepts layer by layer, starting from the Physical layer in Submission 1 and extending to the Data Link layer in Submission 2. All devices, connections, and protocol behaviors are represented as Python classes, enabling realistic simulation of packet transmission, error control, flow control, and address learning without any real network hardware.

Submission 1 – Physical Layer

Objective

Implement the Physical layer of the protocol stack, enabling the creation of end devices and hubs, formation of network topologies, and simulation of data transmission.

Implemented Components

Packet	Simple data container holding source, destination, and payload fields.
--------	--

EndDevice	Models a PC or phone. Supports bidirectional connections, broadcast send (shared media), direct send (point-to-point), and separate receive methods for each mode.
Hub	Layer 1 device. Receives a frame and floods it to all ports except the incoming port — faithfully simulating real hub behavior.

Test Cases

Test 1 – Direct P2P Connection: Two EndDevices connected via a dedicated link. `pc1.send_direct(packet, pc2)` delivers the frame exclusively to PC2 without any broadcast. Verified that PC1's inbox remains empty (no loopback).

Test 2 – Star Topology with Hub: Five PCs connected to a single Hub. PC1 sends a broadcast-destined frame to PC3. Hub floods the frame to all four other ports; only PC3 accepts it (destination match). PC2, PC4, PC5 discard it.

Test 3 – Direct Send in Hub Topology (negative test): Demonstrates that `send_direct()` correctly fails when the target device is not directly connected — enforcing correct method selection per topology.

Submission 2 – Data Link Layer

Objective

Extend the simulator with Layer 2 functionality including bridge and switch devices, MAC address learning, error control, access control (CSMA/CD), and sliding-window flow control.

Implemented Components

Frame	Encapsulates source MAC, destination MAC, sequence number, payload, and a CRC-like checksum for error detection.
Switch	Layer 2 device implementing MAC address table learning. Forwards unicast frames only to the learned port; floods unknown destinations. Tracks broadcast and collision domains.
Bridge	Connects two network segments. Learns MAC addresses per segment and selectively forwards or filters frames, reducing unnecessary traffic.
Error Control	CRC-based error detection. Frames with corrupted checksums are dropped, and a NAK is returned to the sender to trigger retransmission.
Access Control	CSMA/CD protocol — devices sense the channel before transmitting; collisions are detected and resolved via exponential back-off.
Flow Control	Go-Back-N sliding window protocol — sender maintains a window of unacknowledged frames; on timeout or NAK, the entire window is retransmitted from the lost frame.

Test Cases

Test 4 – Switch with Five PCs: Five EndDevices connected to a Switch. Initially all frames are flooded (unknown MACs). After a few exchanges, the switch's MAC table is populated and subsequent frames are forwarded only to the correct port. Broadcast and collision domains are reported: 1 broadcast domain, 5 collision domains (one per port).

Test 5 – Two Hub-based Stars Bridged by a Switch: Two groups of five PCs each connected to their own Hub; the two hubs are connected via a Switch. Communication between all 10 devices is demonstrated. Result: 2 broadcast domains (one per hub segment), 3 collision domains (two hub segments + the switch link).

Test 6 – Go-Back-N Flow Control Demo: Sender transmits a sequence of frames with window size N. Artificial packet loss is introduced mid-sequence to trigger retransmission from the lost frame, verifying correct GBN behaviour.

Test 7 – CSMA/CD Access Control Demo: Multiple devices attempt simultaneous transmission on a shared hub segment. Collision detection fires, back-off is computed, and re-transmission succeeds after the channel clears.

Assumptions

- All devices are uniquely named; MAC addresses are auto-generated 6-byte hex strings.
- Transmission is instantaneous within the simulation (no propagation delay model).
- Error injection (bit flips) is performed programmatically to validate error control logic.
- CSMA/CD back-off uses a simplified random delay bounded by the collision count.
- Sliding window size N is configurable at runtime (default N = 3).
- No physical layer noise model is applied unless explicitly enabled for testing.

How to Run the Code

Requirements: Python 3.8 or higher. No third-party packages required.

1. Clone the repository: `git clone <repo-url>`
2. Navigate to the project directory: `cd network_protocol_simulator`
3. Run Submission 1 tests: `python Semester_Project_part_1.ipynb` (or open in Jupyter)
4. Run Submission 2 tests: `python Semester_Project_part_2.ipynb` (or open in Jupyter)
5. Each test function prints a detailed trace of packet flow, protocol events, and pass/fail results to the console.

Output Format

All output is console-based. Each event is tagged with a bracketed label for easy reading:

[CONNECT]	– Physical link established between two devices.
[BROADCAST SEND / DIRECT SEND]	– Frame transmission initiated.
[HUB]	– Hub flooding event with port-by-port trace.

[SWITCH]	– Forwarding decision (unicast / flood) with MAC table state.
[ERROR]	– Checksum mismatch detected; frame dropped.
[CSMA/CD]	– Collision event and back-off information.
[GBN]	– Go-Back-N window state, ACKs, and retransmissions.
[RESULT]	– Domain count and test pass/fail summary.

Add-ons Implemented

- Topology visualisation: text-based adjacency list printed after every connect() call.
- Additional error detection: parity bit check alongside CRC for single-bit error classification.
- Selective Repeat protocol implemented as an alternative to Go-Back-N.
- Configurable noise model: random bit-flip probability injected per frame for stress testing.

References

- [1] Guidance and mentorship by **Dr Iqra Altaf Gilani**, Department of IT, NIT Srinagar.
- [2] Class notes: Physical Layer, Line Coding, Data Link Layer, and Sliding Window Protocols.
- [3] OpenAI. (2024). *ChatGPT* [Large language model]. Used for code assistance and conceptual clarification. <https://chat.openai.com>
-